Lecture 4: Newton's method, Levenberg–Marquardt, and conditioning

COMP 521 • Probability, Optimization, and Scientific Discovery • Sections 2.5–2.7

This handout is one part of the full COMP 521 notes. Section, equation, figure and listing numbers match the full notes, and references to anything outside this handout point there. The notation table is in "How to use these notes" (Lecture 1).

2.5 Newton, quasi-Newton, Gauss–Newton, Levenberg–Marquardt

Newton's method minimizes the local quadratic model $\mathcal{L}(\boldsymbol{\theta}+\boldsymbol{\Delta}) \approx \mathcal{L} + \mathbf{g}^\mathsf{T}\boldsymbol{\Delta} + \frac{1}{2}\boldsymbol{\Delta}^\mathsf{T}\mathbf{H}\boldsymbol{\Delta}$ by solving

$$\mathbf{H}\boldsymbol{\Delta} = -\mathbf{g}, \qquad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \boldsymbol{\Delta}. \tag{19}$$

Solve the linear system; never form $\mathbf{H}^{-1}$. On a quadratic, Newton converges in one step regardless of κ (last lines of Listing 2). Far from a minimum, $\mathbf{H}$ may be indefinite and the Newton step can point uphill, so practical methods add safeguards:

- **Line search** picks a step length along a direction, accepting it only if the loss falls enough (the Armijo condition $\mathcal{L}(\boldsymbol{\theta}+t\boldsymbol{\Delta}) \le \mathcal{L}(\boldsymbol{\theta}) + c\,t\,\mathbf{g}^\mathsf{T}\boldsymbol{\Delta}$, $c \approx 10^{-4}$).
- **Trust regions** trust the quadratic model only within a radius, which grows when predicted and actual decreases agree and shrinks when they do not.
- **Quasi-Newton** methods (BFGS, L-BFGS) build curvature estimates from successive gradients. They are strong defaults for smooth, deterministic objectives of moderate dimension.
- **Gauss–Newton** replaces $\mathbf{H}$ by $\mathbf{H}_{\mathrm{GN}} = \mathbf{J}^\mathsf{T}\boldsymbol{\Sigma}^{-1}\mathbf{J}$ for least squares.
- **Levenberg–Marquardt** (LM) damps Gauss–Newton,

  $$(\mathbf{J}^\mathsf{T}\boldsymbol{\Sigma}^{-1}\mathbf{J} + \nu\,\mathbf{D})\boldsymbol{\Delta} = -\mathbf{J}^\mathsf{T}\boldsymbol{\Sigma}^{-1}\mathbf{r}, \tag{20}$$

  with $\mathbf{D}$ the identity or $\mathrm{diag}(\mathbf{H}_{\mathrm{GN}})$. Large ν gives a short, scaled gradient step; small ν gives Gauss–Newton. Adjusting ν after each accepted or rejected step is a simple trust-region strategy.

ex04_levenberg_marquardt.py -- LM from scratch on a TOY cooling curve

```python
"""Levenberg-Marquardt from first principles on a toy cooling curve.

Model (toy): y(t; theta) = exp(a) * exp(-t / exp(b)),  theta = (a, b) = (log L0, log tau).
Log parameters keep L0 and tau positive and make their scales comparable.
Shows that J^T Sigma^-1 J is simultaneously the Gauss-Newton Hessian, the Fisher
matrix, and (plus the prior precision) the Laplace-approximation posterior precision.
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)

t = jnp.linspace(0.0, 4.0, 12)
theta_true = jnp.array([jnp.log(2.0), jnp.log(1.5)])
sigma = 0.05 * jnp.ones_like(t)


def model(theta):
    a, b = theta
    return jnp.exp(a) * jnp.exp(-t / jnp.exp(b))


d = model(theta_true) + sigma * jax.random.normal(jax.random.key(3), t.shape)


def residual(theta):  # whitened residuals r / sigma, so Sigma^-1 is the identity below
    return (model(theta) - d) / sigma


def loss(theta):
    r = residual(theta)
    return 0.5 * r @ r


jac = jax.jacfwd(residual)  # 12 x 2 Jacobian of whitened residuals

theta, nu = jnp.array([0.0, 0.0]), 1e-2  # start far off; nu = damping
for it in range(50):
    r, J = residual(theta), jac(theta)
    g, H_gn = J.T @ r, J.T @ J
    step = jnp.linalg.solve(H_gn + nu * jnp.diag(jnp.diag(H_gn)), -g)
    if loss(theta + step) < loss(theta):
        theta, nu = theta + step, nu / 3.0  # accept: trust the quadratic model more
    else:
        nu = nu * 5.0  # reject: damp toward (scaled) gradient descent
    if jnp.linalg.norm(g) < 1e-6:  # small gradient: stop
        break

J = jac(theta)
F = J.T @ J  # Fisher information for Gaussian noise with fixed covariance
H_full = jax.hessian(loss)(theta)
print(f"stopped after {it + 1} iterations: theta = {theta}, truth = {theta_true}")
print(f"chi^2 at optimum = {2 * loss(theta):.2f} for {t.size - 2} degrees of freedom")
print("Gauss-Newton Hessian / Fisher:\n", F)
print("Full Hessian (adds sum_i r_i * Hess(r_i)):\n", H_full)

prior_cov = jnp.diag(jnp.array([1.0, 1.0]))  # N(0, 1) priors on a and b
```

```
post_cov = jnp.linalg.inv(F + jnp.linalg.inv(prior_cov))  # Laplace approximation
print("Laplace posterior sd:", jnp.sqrt(jnp.diag(post_cov)))
print("posterior correlation of (a, b):",
      post_cov[0, 1] / jnp.sqrt(post_cov[0, 0] * post_cov[1, 1]))
```

Output (executed for these notes):

```
stopped after 11 iterations: theta = [0.69581024 0.4158928 ], truth = [0.69314718 0.40546511]
chi^2 at optimum = 7.23 for 10 degrees of freedom
Gauss-Newton Hessian / Fisher:
 [[4207.41458128 1600.84176474]
 [1600.84176474 1530.17117584]]
Full Hessian (adds sum_i r_i * Hess(r_i)):
 [[4207.41458128 1600.84176483]
 [1600.84176483 1490.16330579]]
Laplace posterior sd: [0.0198625  0.03292921]
posterior correlation of (a, b): -0.6306338457586195
```

Read the output closely. The fit recovers the truth to within its uncertainty. $\chi^2 = 7.2$ for 10 degrees of freedom is a reasonable fit. The Gauss–Newton matrix agrees with the full Hessian in the (a, a) and (a, b) entries (to the precision of the stopping rule) and differs in the (b, b) entry by 3%. The exact agreement is a consequence of how the amplitude enters the model, not of luck; Exercise 8 asks you to prove it. The 3% difference is small because the residuals at the optimum are small, which is when Gauss–Newton is a good approximation. The Laplace posterior (previewed here, derived in Section 3.3) shows the two parameters are anticorrelated: a brighter, faster-cooling curve and a fainter, slower one fit nearly equally well.

Other optimizers. *Adam* rescales each coordinate by running estimates of gradient moments; it is designed for noisy mini-batch gradients in neural-network training and is usually *worse* than L-BFGS or LM for smooth stellar-parameter fits. *CMA-ES* adapts a population distribution and needs no gradients, which makes it a robust fallback when gradients are unavailable or untrustworthy, at a cost of many more model evaluations. In the JAX ecosystem, Optimistix provides Newton root-finding, quasi-Newton (BFGS), Gauss–Newton and LM solvers and Optax provides first-order optimizers such as Adam.

Situation	Reasonable first choice	Why
smooth least squares, $p \lesssim 10^3$	LM or Gauss–Newton with trust region	uses residual structure
smooth general loss, moderate p	L-BFGS with line search	curvature from gradients only
huge p, noisy mini-batch gradients	Adam or SGD with momentum	cheap, tolerant of noise
gradients unavailable or wrong	CMA-ES, Nelder–Mead	no derivatives needed
several basins suspected	any of the above from many starts	local methods find local minima

Choose by smoothness, dimension, derivative validity and constraints, then measure convergence. Do not choose by fashion.

2.6 Minima, convexity, and conditioning

A *local minimum* is no worse than nearby feasible points; a *global minimum* is no worse than any feasible point. A function is *convex* if the chord between any two points of its graph lies *on or above*

the graph. For differentiable f on a convex domain this is equivalent to

$$f(\boldsymbol{\theta}') \geq f(\boldsymbol{\theta}) + \nabla f(\boldsymbol{\theta})^{\mathsf{T}}(\boldsymbol{\theta}' - \boldsymbol{\theta}) \quad \text{for all } \boldsymbol{\theta}, \boldsymbol{\theta}', \tag{21}$$

so every tangent plane is a *global* under-estimator. The word "global" is the point: near a strict local minimum, every smooth function satisfies such a bound locally, and that guarantees nothing about other basins. Consequently a convex function on a convex set has no local minima that are not global (the minimizer need not be unique).

Astrophysical losses are usually non-convex, with several basins and long, flat valleys. A tiny gradient may mean a minimum, a saddle, or a direction the data do not constrain at all.

Conditioning measures how much a solution moves when its inputs are perturbed. If two columns of $\mathbf{J}$ are nearly parallel, different parameter combinations produce nearly the same observables, and $\mathbf{J}^{\mathsf{T}}\mathbf{J}$ is ill-conditioned: its condition number is the *square* of that of $\mathbf{J}$, which is one reason good least-squares codes factor $\mathbf{J}$ (QR or SVD) instead of forming $\mathbf{J}^{\mathsf{T}}\mathbf{J}$.

Choose coordinates deliberately. Parameters with very different units or scales should be nondimensionalized, $z_i = (\theta_i - \theta_{0,i})/s_i$, or logged, $z = \log\theta$, for positive quantities (as in Listing 3). This changes the numerical geometry, and when the ill-conditioning comes purely from mismatched scales (as in Listing 2) it can turn a $\kappa = 1000$ problem into $\kappa = 1$. It cannot create information the data lack. When the objective is a log-posterior, which is a density over $\boldsymbol{\theta}$, changing coordinates also adds the log-Jacobian of Eq. (10). A log-likelihood is a density in $\mathbf{d}$, not in $\boldsymbol{\theta}$, so it gets no Jacobian term (Section 3).

Convergence is not validation

Stopping rules (small gradient, small step, small relative decrease) are statements about the algorithm. They certify neither global optimality, nor physical correctness, nor identifiability. Validation means checking numerical accuracy, derivatives, residual structure, physical assumptions, and predictions for data not used in the fit.

2.7 Regularization, priors, and constraints

Regularization adds preferences to a fit, for example $\mathcal{J}(\boldsymbol{\theta}) = \mathcal{L}_{\text{data}}(\boldsymbol{\theta}) + \lambda\|\boldsymbol{\theta}\|_2^2$. The penalty depends on parameter scales and is not a physical law.

There is an exact Bayesian reading. A Gaussian prior $p(\boldsymbol{\theta}) \propto \exp(-\frac{1}{2}\boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{\Sigma}_0^{-1}\boldsymbol{\theta})$ adds $\frac{1}{2}\boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{\Sigma}_0^{-1}\boldsymbol{\theta}$ to the negative log-likelihood, so ridge regression with strength λ is MAP estimation with $\boldsymbol{\Sigma}_0 = (2\lambda)^{-1}\mathbf{I}$. An ℓ_1 penalty $\lambda\sum_i|\theta_i|$ is MAP with a Laplace prior. Note the asymmetry: the ℓ_1 *MAP estimate* is often exactly sparse, but the posterior *mean* under the same prior is not. "Regularization" and "prior" are the same mathematics; writing the prior makes the assumption visible and lets you propagate it.

Hard constraints have their own toolbox. *Projection* maps each iterate back into the feasible set (easy for bounds). *Reparameterization* removes the constraint ($f_{\text{ov}} = e^z$; a softmax for mass fractions). It has a cost: e^z approaches the bound but never reaches it, so if the constraint is active at the optimum ($f_{\text{ov}}^\star = 0$), the optimizer drifts toward $z \to -\infty$ instead of stopping. *Barrier* methods add terms that blow up at the boundary. *Lagrange multipliers* turn constraints into stationarity conditions (the KKT conditions for inequalities). *ADMM* splits certain structured problems into

easier subproblems; it carries convergence guarantees for convex problems, not for nonconvex coupled physics.

Exercises for this lecture

Exercises 8, 9, numbered as in Appendix A of the full notes. Worked solutions are in Appendix B; attempt each one before reading its solution.

8. [P] Why the Gauss–Newton matrix is exact here. In Listing 3, the Gauss–Newton matrix equals the full Hessian in the (a, a) and (a, b) entries at the optimum, and differs only in (b, b). Prove this. (Hint: compute $\partial^2 r_i/\partial a^2$ and $\partial^2 r_i/\partial a\,\partial b$ for $r_i \propto e^a e^{-t_i/e^b} - d_i$, and use the stationarity condition.)

9. [C] A noise model that matters. Modify Listing 3 to fit Poisson counts $n_i \sim \text{Poisson}(\mu_i)$ with $\mu_i = e^a e^{-t_i/e^b}$ and $e^a = 8$, using the Poisson negative log-likelihood. Fit the same data with χ^2 and $\sigma_i = \sqrt{n_i}$. Repeat over 500 simulated data sets and compare the bias of $\hat{a}$ under the two losses. Explain the sign of the χ^2 bias.